TCS NQT DIGITAL CODING ROUND

Complete Documentation & Study Textbook

Topics Covered

Arrays · Strings · Numbers · Number Systems · Sorting · Matrices · Collections

Each Chapter Includes:

Concept Explanation · Step-by-Step Algorithms · Java Code
Built-in Utilities · Tricks & Edge Cases
Worked Sample Questions · Practice Questions · Full Solutions

Language: Java (primary) · Logic applicable to C++ and Python
Difficulty Level: TCS NQT Digital — Easy to Medium

Edition 2024–25

Introduction: How to Use This Book

This book is designed as a complete, self-contained preparation guide for the TCS NQT Digital Coding Round. Unlike a mere list of solved problems, it teaches you the underlying principles so you can adapt to any variation of a question on exam day. Every section follows the same structure: first you learn the concept, then you see it in action with worked examples, and finally you test yourself with practice problems before checking your solutions.

About the TCS NQT Digital Coding Round

The TCS National Qualifier Test (NQT) Digital track includes a Coding Round with two programming problems to be solved in 60 minutes. The difficulty ranges from Easy to Medium on competitive programming scales. Problems are drawn from a fixed set of topic areas, making targeted preparation highly effective.

Attribute	Details
Number of problems	2 questions
Time limit	60 minutes
Scoring	Partial marks awarded (test cases passed)
Languages	Java, C, C++, Python, Perl
Key topics	Arrays, Strings, Math, Sorting, Number Systems, Matrices, Collections

How Each Chapter is Structured

- Concept Explanation — plain-English description of the topic and its real use.
- Core Theory & Algorithms — step-by-step logic for every major method.
- Java Code with Commentary — production-quality, heavily commented code.
- Built-in Utility Reference — Java standard library methods for each topic.
- Tricks, Tips & Edge Cases — what candidates miss in the exam.
- Sample Questions with Answers — immediately see concepts applied.
- Practice Questions (no answers) — attempt independently to build confidence.
- Solutions — verify your work and learn from any gaps.

Language Notes

All code in this book is written in Java, the most widely chosen language for TCS NQT. After each major algorithm, key translation notes for C++ and Python are provided in highlighted boxes. The core logic is identical in all three languages — only syntax and library names change.

JAVA IMPORT CHEAT SHEET

```
import java.util.*;          // Scanner, Arrays, ArrayList, HashMap, etc.
import java.util.Arrays;     // Arrays.sort(), Arrays.fill(), Arrays.copyOf()
import java.util.Collections;// Collections.sort(), Collections.reverse()
import java.math.BigInteger; // for very large factorials
import java.util.Scanner;    // standard input reading
```

Recommended Study Schedule

Day	Chapters	Focus
Day 1	Arrays	All array algorithms, two-pointer, prefix sum
Day 2	Numbers	Math problems, prime, Armstrong, series
Day 3	Number Systems	Binary, octal, conversions, number-to-words
Day 4	Sorting	All 5 sort algorithms, when to use each
Day 5	Strings	Manipulation, frequency, anagram, palindrome
Day 6	Matrices	2D arrays, traversal, rotation, transpose
Day 7	Collections	ArrayList, HashMap, Stack, Queue, PriorityQueue
Day 8	Full Revision	Work through all practice problems under timed conditions

EXAM STRATEGY

1. Read both problems first. Pick the one you can fully solve — do it first.
2. A 100% brute-force solution is better than a 0% optimised attempt.
3. Always handle edge cases: empty input, n=1, all-same elements, negatives.
4. Use Java's built-in sort (Arrays.sort) unless asked to implement an algorithm.
5. If you are stuck, write as much correct code as possible — partial marks count.
6. Dry-run your solution mentally on the given sample input before submitting.

Chapter 1: Arrays

1.1 What is an Array?

📖 DEFINITION

An array is a fixed-size, ordered collection of elements of the same data type stored in contiguous
memory locations. Elements are accessed by their zero-based index in O(1) time.

Think of an array like a row of numbered lockers: every locker has a unique number starting from 0, every locker can hold exactly one item of the same type, and you can jump to any locker instantly if you know its number.

Arrays are the foundation of nearly every data structure and algorithm. In TCS NQT, at least one of the two problems will involve array manipulation, so mastering this chapter is the single highest-return investment you can make.

1.2 Array Basics in Java

```
// ── DECLARATION & INITIALISATION ──────────────────────────────
int[] arr = new int[5];            // {0,0,0,0,0}  (default 0 for int)
int[] arr = {10, 30, 20, 50, 40}; // literal initialisation
int[] arr = new int[]{1, 2, 3};    // anonymous array

// ── LENGTH ────────────────────────────────────────────────────
int n = arr.length;                // 5  (field, not method — no parentheses)

// ── ACCESS ────────────────────────────────────────────────────
arr[0]          // first element
arr[n-1]        // last element

// ── TRAVERSAL ─────────────────────────────────────────────────
for (int i = 0; i < arr.length; i++)  System.out.print(arr[i] + " ");
for (int x : arr)                      System.out.print(x + " ");  // enhanced for

// ── USEFUL METHODS ────────────────────────────────────────────
import java.util.Arrays;
Arrays.sort(arr);                  // ascending sort — O(n log n)
Arrays.sort(arr, 2, 5);            // sort indices 2,3,4 only
Arrays.fill(arr, 0);              // fill entire array with 0
Arrays.copyOf(arr, 3);            // copy first 3 elements to new array
Arrays.copyOfRange(arr, 1, 4);    // copy indices 1,2,3
Arrays.toString(arr);             // "[10, 30, 20, 50, 40]"
Arrays.equals(a, b);              // true if same length and same elements
Arrays.binarySearch(arr, 30);     // index of 30 (array must be sorted first!)
```

🔗 C++ / PYTHON EQUIVALENTS

C++: vector<int> v = {10,30,20}; sort(v.begin(),v.end()); v.size();
Python: arr = [10, 30, 20] arr.sort() len(arr)
Python also: sorted(arr) returns a new sorted list (non-destructive).

1.3 Core Algorithms

1.3.1 Finding Minimum and Maximum

Strategy: Walk through the array once, comparing every element against the running best. Time: O(n), Space: O(1).

Initialise min/max to the first element (not to Integer.MIN_VALUE or 0) because the array may contain only negative numbers.

```
public static int[] minMax(int[] arr) {
    int min = arr[0], max = arr[0];
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] < min) min = arr[i];
        if (arr[i] > max) max = arr[i];
    }
    return new int[]{min, max};
}
// minMax({5,2,8,1,9,3}) -> {1, 9}
```

1.3.2 Second Smallest and Second Largest (Single Pass)

Track four variables: min1, min2, max1, max2. Update all in one loop. Handle duplicates by requiring strict inequality when applicable.

```
public static void secondMinMax(int[] arr) {
    int min1=arr[0], min2=Integer.MAX_VALUE;
    int max1=arr[0], max2=Integer.MIN_VALUE;

    for (int x : arr) {
        // Second minimum
        if (x < min1)                   { min2 = min1; min1 = x; }
        else if (x < min2 && x != min1)  { min2 = x; }

        // Second maximum
        if (x > max1)                   { max2 = max1; max1 = x; }
        else if (x > max2 && x != max1)  { max2 = x; }
    }
    System.out.println("2nd Smallest: " + min2);
    System.out.println("2nd Largest:  " + max2);
}
// {12,13,1,10,34,1} -> 2nd Smallest: 10  2nd Largest: 13
```

DUPLICATE GUARD

The guard (x != min1) ensures we find the second DISTINCT minimum.
Remove it if duplicates are treated as valid second-smallest candidates.
Example: {5,5,5} with guard -> 2nd min = MAX_VALUE (none found).
{5,5,5} without guard -> 2nd min = 5 (same as min1).

1.3.3 Reversing an Array — Two-Pointer

Pattern: TWO POINTER. One pointer starts at index 0, the other at index n-1. Swap and converge.

```
public static void reverse(int[] arr) {
    int left = 0, right = arr.length - 1;
    while (left < right) {
        // Standard swap using temp variable
        int temp   = arr[left];
        arr[left]  = arr[right];
        arr[right] = temp;
        left++;
        right--;
    }
}
// {1,2,3,4,5} -> {5,4,3,2,1}
// {1,2,3,4}   -> {4,3,2,1}
```

1.3.4 Frequency Count — Hashing

Pattern: HASHING. Use a HashMap<Integer,Integer> to map each element to its count. One scan: O(n) time, O(n) space.

```
import java.util.*;

public static Map<Integer,Integer> frequency(int[] arr) {
    Map<Integer,Integer> freq = new LinkedHashMap<>(); // preserves insertion
order
    for (int x : arr)
        freq.put(x, freq.getOrDefault(x, 0) + 1);
    return freq;
}
// {2,3,2,5,3,3,1} -> {2=2, 3=3, 5=1, 1=1}

// Alternatively, for small non-negative integers, use an int array:
int[] count = new int[max + 1];   // index = element, value = count
for (int x : arr) count[x]++;
```

1.3.5 Removing Duplicates

From a SORTED array (in-place, O(1) extra space) — Two-pointer:

```
public static int removeDupsSorted(int[] arr) {
    if (arr.length == 0) return 0;
    int slow = 0;                        // slow pointer: last unique index
    for (int fast = 1; fast < arr.length; fast++) {
        if (arr[fast] != arr[slow]) {    // new unique element found
            slow++;
            arr[slow] = arr[fast];
        }
    }
    return slow + 1;                     // new logical length
}
// {1,1,2,2,3,4,4} -> first 4 elements: {1,2,3,4}, returns 4
```

From an UNSORTED array — LinkedHashSet (preserves order):

```
import java.util.*;

public static int[] removeDupsUnsorted(int[] arr) {
    Set<Integer> seen = new LinkedHashSet<>();
    for (int x : arr) seen.add(x);       // duplicates silently ignored
    // Convert back to array
    return seen.stream().mapToInt(i->i).toArray();
}
// {3,1,4,1,5,9,2,6,5,3} -> {3,1,4,5,9,2,6}
```

1.3.6 Array Rotation — Three-Reversal Trick

Rotating an array left by k positions means elements at indices 0..k-1 move to the end.

The Three-Reversal Trick: reverse[0..k-1] → reverse[k..n-1] → reverse[0..n-1].

This runs in O(n) time and O(1) space — no temporary array needed.

```
private static void reverseRange(int[] a, int l, int r) {
    while (l < r) { int t=a[l]; a[l]=a[r]; a[r]=t; l++; r--; }
}

public static void leftRotate(int[] arr, int k) {
    int n = arr.length;
    k = k % n;                // handle k >= n
    if (k == 0) return;
    reverseRange(arr, 0, k-1);
    reverseRange(arr, k, n-1);
    reverseRange(arr, 0, n-1);
}

public static void rightRotate(int[] arr, int k) {
    leftRotate(arr, arr.length - k % arr.length);
}
```

```
// {1,2,3,4,5} left-rotate by 2  ->  {3,4,5,1,2}
// {1,2,3,4,5} right-rotate by 2 ->  {4,5,1,2,3}
```

1.3.7 Prefix Sum — Range Query in O(1)

Pattern: PREFIX SUM. Build a prefix[] array where prefix[i] = sum of arr[0..i-1]. Then any range sum from index l to r = prefix[r+1] - prefix[l].

```
int[] buildPrefix(int[] arr) {
    int[] prefix = new int[arr.length + 1]; // prefix[0] = 0
    for (int i = 0; i < arr.length; i++)
        prefix[i+1] = prefix[i] + arr[i];
    return prefix;
}

// Sum from index l to r (inclusive):
int rangeSum(int[] prefix, int l, int r) {
    return prefix[r+1] - prefix[l];
}

// arr = {1,2,3,4,5}
// prefix = {0,1,3,6,10,15}
// rangeSum(prefix, 1, 3) = prefix[4]-prefix[1] = 10-1 = 9  (2+3+4)
```

1.3.8 Equilibrium Index

Index i is equilibrium if sum(arr[0..i-1]) == sum(arr[i+1..n-1]). Use prefix sum pattern.

```
public static int equilibriumIndex(int[] arr) {
    long total = 0;
    for (int x : arr) total += x;

    long leftSum = 0;
    for (int i = 0; i < arr.length; i++) {
        long rightSum = total - leftSum - arr[i];
        if (leftSum == rightSum) return i;
        leftSum += arr[i];
    }
    return -1; // no equilibrium index found
}
// {-7,1,5,2,-4,3,0} -> index 3  (leftSum=-1, rightSum=-1)
```

1.3.9 Maximum Product Subarray (Kadane Variant)

Because negative × negative = positive, track BOTH maxProduct and minProduct at every step. Either one multiplied by the current element may become the new maximum.

```
public static int maxProductSubarray(int[] arr) {
```

```
    int maxP = arr[0], minP = arr[0], result = arr[0];
    for (int i = 1; i < arr.length; i++) {
        int x = arr[i];
        int tempMax = Math.max(x, Math.max(maxP * x, minP * x));
        minP        = Math.min(x, Math.min(maxP * x, minP * x));
        maxP        = tempMax;
        result      = Math.max(result, maxP);
    }
    return result;
}
// {2,3,-2,4}    -> 6   (subarray {2,3})
// {-2,0,-1}     -> 0   (subarray {0})
// {-2,-3,-4}    -> 12  (subarray {-3,-4})
```

1.3.10 Replace Element by Rank

Clone and sort the array. Map element → rank using a HashMap (equal elements share the same rank). Replace in original.

```
public static void replaceByRank(int[] arr) {
    int[] sorted = arr.clone();
    Arrays.sort(sorted);
    Map<Integer,Integer> rank = new HashMap<>();
    int r = 1;
    for (int x : sorted)
        if (!rank.containsKey(x)) rank.put(x, r++);
    for (int i = 0; i < arr.length; i++) arr[i] = rank.get(arr[i]);
}
// {20,15,26,2,98,6}  ->  {4,3,5,1,6,2}
```

1.3.11 Sort by Frequency

```
import java.util.*;

public static void sortByFrequency(int[] arr) {
    Map<Integer,Integer> freq = new LinkedHashMap<>();
    for (int x : arr) freq.put(x, freq.getOrDefault(x,0)+1);

    List<Integer> keys = new ArrayList<>(freq.keySet());
    // Sort descending by frequency; tie-break ascending by value
    keys.sort((a,b) -> freq.get(b)!=freq.get(a) ?
              freq.get(b)-freq.get(a) : a-b);

    for (int k : keys)
        for (int i = 0; i < freq.get(k); i++)
            System.out.print(k + " ");
}
// {2,3,2,4,5,12,2,3,3,3,12} -> 3 3 3 3 2 2 2 12 12 4 5
```

1.3.12 Subset Check

```
public static boolean isSubset(int[] A, int[] B) {
    // Check if B is a subset of A
    Set<Integer> setA = new HashSet<>();
    for (int x : A) setA.add(x);
    for (int x : B) if (!setA.contains(x)) return false;
    return true;
}
// isSubset({11,1,13,21,3,7}, {11,3,7,1}) -> true
```

1.3.13 Find All Symmetric Pairs

(a,b) and (b,a) are symmetric. Use a HashMap where key=first, value=second. For each pair (x,y), check if map contains y as key with value x.

```
public static void symmetricPairs(int[][] pairs) {
    Map<Integer,Integer> map = new HashMap<>();
    for (int[] p : pairs) {
        int x=p[0], y=p[1];
        if (map.containsKey(y) && map.get(y)==x)
            System.out.println("("+x+","+y+") and ("+y+","+x+")");
        else map.put(x, y);
    }
}
// {{1,2},{3,4},{2,1},{4,3}} -> (2,1) and (1,2),  (4,3) and (3,4)
```

1.3.14 Sort According to Another Array's Order

```
public static void sortByOrder(int[] A, int[] B) {
    Map<Integer,Integer> freq = new HashMap<>();
    for (int x : A) freq.put(x, freq.getOrDefault(x,0)+1);

    List<Integer> result = new ArrayList<>();
    // Add B's elements first (in B's order)
    for (int b : B) {
        if (freq.containsKey(b)) {
            for (int i=0; i<freq.get(b); i++) result.add(b);
            freq.remove(b);
        }
    }
    // Remaining elements sorted ascending
    List<Integer> rest = new ArrayList<>(freq.keySet());
    Collections.sort(rest);
    for (int k : rest) for (int i=0; i<freq.get(k); i++) result.add(k);
    System.out.println(result);
}
```

```
// A={2,1,2,5,7,1,9,3}, B={2,1,8,3} -> [2,2,1,1,3,5,7,9]
```

1.3.15 Binary Search

Requires a SORTED array. Divides search space in half each step: O(log n). NEVER write mid = (l+r)/2 — use l + (r-l)/2 to avoid integer overflow.

```
public static int binarySearch(int[] arr, int target) {
    int l = 0, r = arr.length - 1;
    while (l <= r) {
        int mid = l + (r - l) / 2;          // overflow-safe midpoint
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) l = mid + 1;
        else                        r = mid - 1;
    }
    return -1;  // not found
}
// Built-in: Arrays.binarySearch(arr, target)
// Returns negative value if not found — always check: result >= 0
```

1.4 Tricks, Tips & Edge Cases

⚠ CRITICAL EDGE CASES FOR ARRAYS

Empty array: Always check arr.length == 0 before accessing arr[0].
Single element: Many algorithms have special behaviour for n=1.
All equal elements: {5,5,5} — second smallest/largest may not exist.
All negative: Min/max must be initialised to arr[0], not 0.
Overflow in sum: Use long instead of int when summing large arrays.
Overflow in product: Cast to long before multiplying large int values.
k >= n in rotation: Always use k = k % n to normalise.

Problem Pattern	Best Data Structure / Technique
Find min/max/sum	Linear scan, O(n)
Duplicates / frequency	HashMap or int[] for small values
Pair with given sum	Sorted + two-pointer OR HashMap
Subarray sum queries	Prefix sum array
In-place modification	Two-pointer (avoid extra space)
Sorted input	Binary search for O(log n) lookup
Order preservation with no dups	LinkedHashSet

🔍 WORKED EXAMPLES — Sample Questions & Answers

Q1: Given arr={3,1,4,1,5,9,2,6,5}, find frequency of each element.

Answer: Use HashMap. Scan once.

```
int[] arr = {3,1,4,1,5,9,2,6,5};
Map<Integer,Integer> freq = new LinkedHashMap<>();
for (int x : arr) freq.put(x, freq.getOrDefault(x,0)+1);
for (var e : freq.entrySet())
    System.out.println(e.getKey() + " -> " + e.getValue());
// Output: 3->1, 1->2, 4->1, 5->2, 9->1, 2->1, 6->1
```

Q2: Find equilibrium index of {1,3,5,2,2}.

Answer: totalSum=13. At i=0: left=0, right=12. At i=1: left=1, right=9. At i=2: left=4, right=4. Match! Index=2.

Q3: Rotate {1,2,3,4,5} left by 2. Show step-by-step.

Answer: Step1: reverse[0..1]: {2,1,3,4,5}. Step2: reverse[2..4]: {2,1,5,4,3}. Step3: reverse all: {3,4,5,1,2}.

Q4: Find the max product subarray of {-2,-3,0,-2,-40}.

Answer: Track maxP and minP. At i=4(x=-40): tempMax=max(-40,(-2)*(-40),(0)*(-40))=max(-40,80,0)=80. Result=80. Subarray={-2,-40}.

✏ PRACTICE QUESTIONS — Try These Yourself

1. Given arr={5,3,8,1,9,2,7}, write code to output all non-repeating elements.
2. Rotate {10,20,30,40,50} right by 3 positions. What is the result?
3. Check if {3,6} is a subset of {1,2,3,4,5,6,7}. What is the time complexity of your approach?
4. Find the second largest element in {10,5,10,6,3,10}. What is it?
5. Given {2,4,6,8,10}, find the sum of elements from index 1 to 3 using prefix sum.
6. Sort {3,1,2,1,3,3,2} by descending frequency. What is the sorted output?
7. Find all symmetric pairs in {{11,20},{30,40},{20,11},{5,10},{40,30}}.

✔ SOLUTIONS TO PRACTICE QUESTIONS

Solution 1: Non-repeating elements

```
// All elements are unique -> output: 5 3 8 1 9 2 7
```

Solution 2: Right rotate {10,20,30,40,50} by 3 -> {30,40,50,10,20}

Right rotate by 3 = left rotate by (5-3)=2. {30,40,50,10,20}

Solution 3: Put {1..7} in HashSet, check 3 and 6. O(n+m) time.

Solution 4: Second largest in {10,5,10,6,3,10} — max1=10, max2=6. Answer: 6.

Solution 5: prefix={0,2,6,12,20,30}. rangeSum(1,3)=prefix[4]-prefix[1]=20-2=18.

Solution 6: freq: 3->3,1->2,2->2. By desc freq: 3 3 3 1 1 2 2.

Solution 7: (11,20)&(20,11) are symmetric; (30,40)&(40,30) are symmetric.

Chapter 2: Numbers

Number-based problems in TCS NQT test your mathematical reasoning and ability to translate formulas into code. They typically require less than 30 lines of code once you know the pattern. This chapter covers every number-classification and number-computation problem that appears in the TCS NQT coding sheet.

2.1 Digit Extraction — The Foundation

Most number problems work by processing digits one at a time. The standard technique extracts digits from right to left using modulo and integer division.

```
int n = 12345;
while (n > 0) {
    int digit = n % 10;   // rightmost digit: 5, then 4, then 3, ...
    n /= 10;              // remove that digit
}

// Useful helpers:
int numDigits = (int) Math.log10(n) + 1;   // number of digits in n
int numDigits = Integer.toString(n).length(); // string-based alternative
```

2.2 Number Classification Problems

2.2.1 Palindrome Number

📖 DEFINITION

A number is a palindrome if it reads the same forwards and backwards.

```
public static boolean isPalindrome(int n) {
    if (n < 0) return false;        // negative numbers are never palindromes
    int original = n, reversed = 0;
    while (n > 0) {
        reversed = reversed * 10 + n % 10;
        n /= 10;
    }
    return original == reversed;
}
// isPalindrome(121) -> true   isPalindrome(123) -> false

// Find all palindromes in range [low, high]:
for (int i = low; i <= high; i++)
    if (isPalindrome(i)) System.out.print(i + " ");
```

2.2.2 Prime Number

> 📖 **DEFINITION**
> A prime has exactly two factors: 1 and itself. Primes: 2,3,5,7,11,13,...

Key insight: if n has a factor d > sqrt(n), then n/d < sqrt(n) is also a factor. So we only need to test divisors up to sqrt(n).

```
public static boolean isPrime(int n) {
    if (n < 2) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;        // even > 2 -> not prime
    for (int i = 3; (long)i*i <= n; i += 2) { // odd divisors only
        if (n % i == 0) return false;
    }
    return true;
}

// ── SIEVE OF ERATOSTHENES — all primes up to 'limit' ──────────
// Time: O(n log log n)   Space: O(n)
public static boolean[] sieve(int limit) {
    boolean[] isPrime = new boolean[limit + 1];
    Arrays.fill(isPrime, true);
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; (long)i*i <= limit; i++)
        if (isPrime[i])
            for (int j = i*i; j <= limit; j += i)
                isPrime[j] = false;
    return isPrime;
}
// All primes up to 50: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

2.2.3 Armstrong Number

> 📖 **DEFINITION**
> Sum of each digit raised to the power of (number of digits) = the number itself.

Examples: 153 = $1^3+5^3+3^3$ = 1+125+27 = 153. 370 = $3^3+7^3+0^3$ = 27+343+0 = 370.

```
public static boolean isArmstrong(int n) {
    String s  = Integer.toString(n);
    int power = s.length();           // number of digits
    int sum   = 0, temp = n;
    while (temp > 0) {
        int d = temp % 10;
        sum += (int) Math.pow(d, power);
        temp /= 10;
    }
    return sum == n;
}
// Armstrong 3-digit: 153, 370, 371, 407
// Armstrong 4-digit: 1634, 8208, 9474
```

2.2.4 Perfect Number

> **📖 DEFINITION**
> A perfect number equals the sum of its proper divisors (all divisors except itself).

Examples: 6 = 1+2+3. 28 = 1+2+4+7+14. 496. 8128.

```
public static boolean isPerfect(int n) {
    if (n <= 1) return false;
    int sum = 1;                          // 1 is always a proper divisor
    for (int i = 2; (long)i*i <= n; i++) {
        if (n % i == 0) {
            sum += i;
            if (i != n/i) sum += n/i;   // avoid counting sqrt twice
        }
    }
    return sum == n;
}
```

2.2.5 Strong Number

> **📖 DEFINITION**
> Sum of factorials of digits = the number itself.

Known strong numbers: 1, 2, 145, 40585.

```
static int factorial(int d) {
    int f = 1;
    for (int i = 2; i <= d; i++) f *= i;
    return f;
}

public static boolean isStrong(int n) {
    int sum = 0, temp = n;
    while (temp > 0) { sum += factorial(temp % 10); temp /= 10; }
    return sum == n;
}
// isStrong(145) -> 1!+4!+5! = 1+24+120 = 145 -> true
```

2.2.6 Automorphic Number

> **📖 DEFINITION**
> n^2 ends with n itself.

```
public static boolean isAutomorphic(long n) {
    long sq   = n * n;
    String s  = Long.toString(n);
```

```
    String sq_s = Long.toString(sq);
    return sq_s.endsWith(s);
}
// 5 -> 25 ends in 5 -> true
// 25 -> 625 ends in 25 -> true
// 376 -> 141376 ends in 376 -> true
```

2.2.7 Harshad (Niven) Number

📖 **DEFINITION**

A number divisible by the sum of its own digits.

```
public static boolean isHarshad(int n) {
    int sum = 0, temp = n;
    while (temp > 0) { sum += temp % 10; temp /= 10; }
    return n % sum == 0;
}
// isHarshad(18) -> digit sum=9, 18%9=0 -> true
// isHarshad(19) -> digit sum=10, 19%10=9 -> false
```

2.2.8 Abundant Number

📖 **DEFINITION**

Sum of proper divisors > n.

```
public static boolean isAbundant(int n) {
    int sum = 1;
    for (int i = 2; (long)i*i <= n; i++)
        if (n%i==0) { sum+=i; if(i!=n/i) sum+=n/i; }
    return sum > n;
}
// isAbundant(12) -> divisors {1,2,3,4,6} sum=16 > 12 -> true
```

2.2.9 Even / Odd and Positive / Negative

```
boolean isEven = (n % 2 == 0);
boolean isOdd  = (n % 2 != 0);

// Works for negative numbers too:
// -5 % 2 = -1 in Java (not 1), so use Math.abs if needed:
boolean isEven2 = (Math.abs(n) % 2 == 0);

boolean isPositive = (n > 0);
boolean isNegative = (n < 0);
boolean isZero     = (n == 0);
```

2.2.10 Leap Year

```
// Leap year: divisible by 4, EXCEPT centuries (div by 100),
//            UNLESS also divisible by 400.
public static boolean isLeapYear(int year) {
    return (year % 4 == 0) && (year % 100 != 0 || year % 400 == 0);
}
// 2000 -> true (div by 400)  1900 -> false (div by 100, not 400)  2024 -> true
```

2.3 GCD, LCM, and Related

2.3.1 GCD — Euclidean Algorithm

gcd(a,b) = gcd(b, a%b). When b=0, gcd=a. O(log min(a,b)).

```
public static int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}
// gcd(12,18) = gcd(18,12) = gcd(12,6) = gcd(6,0) = 6
```

2.3.2 LCM

```
public static long lcm(int a, int b) {
    // Divide BEFORE multiplying to avoid overflow
    return (long) a / gcd(a, b) * b;
}
// lcm(4,6) = 4/2 * 6 = 12
```

2.3.3 Prime Factors

```
public static void primeFactors(int n) {
    for (int i = 2; (long)i*i <= n; i++) {
        while (n % i == 0) {
            System.out.print(i + " ");
            n /= i;
        }
    }
    if (n > 1) System.out.print(n); // remaining prime factor
}
// primeFactors(360) -> 2 2 2 3 3 5
```

2.3.4 All Factors of a Number

```
public static void factors(int n) {
    for (int i = 1; (long)i*i <= n; i++) {
        if (n % i == 0) {
            System.out.print(i + " ");
```

```
            if (i != n/i) System.out.print(n/i + " ");
        }
    }
}
// factors(36) -> 1 36 2 18 3 12 4 9 6 (in unsorted order)
```

2.4 Mathematical Computations

2.4.1 Fibonacci Sequence

```
// Iterative — O(n) time, O(1) space  [PREFERRED]
public static void printFib(int n) {
    long a = 0, b = 1;
    System.out.print(a + " " + b);
    for (int i = 2; i < n; i++) {
        long c = a + b;
        System.out.print(" " + c);
        a = b; b = c;
    }
}
// printFib(10) -> 0 1 1 2 3 5 8 13 21 34

// Recursive — DO NOT use for n>40 (exponential time)
long fib(int n) { return n<=1 ? n : fib(n-1)+fib(n-2); }
```

⚠ OVERFLOW WARNING

fib(46)=1,836,311,903 is near int limit (2^31-1 = 2,147,483,647).
fib(47) overflows int. Always use long for Fibonacci.
fib(93) overflows long. Use BigInteger for n>92.

2.4.2 Factorial

```
public static long factorial(int n) {
    long result = 1;
    for (int i = 2; i <= n; i++) result *= i;
    return result;
}
// factorial(10) = 3628800  (fits in long up to n=20)

// BigInteger for large n:
import java.math.BigInteger;
public static BigInteger factorialBig(int n) {
    BigInteger r = BigInteger.ONE;
    for (int i = 2; i <= n; i++) r = r.multiply(BigInteger.valueOf(i));
    return r;
}
```

2.4.3 Power (Exponentiation by Squaring)

```
// Fast power: O(log exp)  — much faster than naive loop
public static long power(long base, int exp) {
    long result = 1;
    while (exp > 0) {
        if ((exp & 1) == 1) result *= base;  // exp is odd
        base *= base;
        exp >>= 1;                           // exp /= 2
    }
    return result;
}
// power(2,10) = 1024    Math.pow(2,10) = 1024.0 (double)
```

2.4.4 Sum of AP and GP Series

```
// AP: a, a+d, a+2d, ...  Sum(n) = n/2 * (2a + (n-1)d)
public static double sumAP(double a, double d, int n) {
    return n / 2.0 * (2*a + (n-1)*d);
}
// sumAP(1, 2, 5) = 5/2*(2+8) = 25   [1+3+5+7+9]

// GP: a, ar, ar², ...   Sum(n) = a*(r^n - 1)/(r-1) for r!=1
public static double sumGP(double a, double r, int n) {
    if (r == 1) return n * a;
    return a * (Math.pow(r, n) - 1) / (r - 1);
}
// sumGP(1, 2, 5) = 1*(32-1)/1 = 31  [1+2+4+8+16]
```

2.4.5 Reverse Digits of a Number

```
public static int reverseDigits(int n) {
    boolean neg = n < 0;
    if (neg) n = -n;
    int rev = 0;
    while (n > 0) { rev = rev*10 + n%10; n /= 10; }
    return neg ? -rev : rev;
}
// reverseDigits(12345) = 54321
// reverseDigits(-456)  = -654
```

2.4.6 Sum of Digits

```
public static int sumDigits(int n) {
    n = Math.abs(n);
    int sum = 0;
    while (n > 0) { sum += n%10; n /= 10; }
    return sum;
```

```
}
// sumDigits(12345) = 15  sumDigits(-999) = 27
```

2.4.7 Max and Min Digit in a Number

```
public static int[] maxMinDigit(int n) {
    n = Math.abs(n);
    int maxD = 0, minD = 9;
    while (n > 0) {
        int d = n % 10;
        if (d > maxD) maxD = d;
        if (d < minD) minD = d;
        n /= 10;
    }
    return new int[]{maxD, minD};
}
// maxMinDigit(382651) -> max=8, min=1
```

2.4.8 Permutations P(n,r)

```
// P(n,r) = n!/(n-r)! = n*(n-1)*...*(n-r+1)  — r terms
public static long permutations(int n, int r) {
    if (r > n) return 0;
    long result = 1;
    for (int i = n; i > n-r; i--) result *= i;
    return result;
}
// P(5,2) = 5*4 = 20  (5 people, 2 seats)
```

2.4.9 Adding Two Fractions

```
// a/b + c/d = (a*d + b*c) / (b*d), then reduce by GCD
public static void addFractions(int a,int b,int c,int d) {
    int num = a*d + b*c, den = b*d;
    int g = gcd(Math.abs(num), Math.abs(den));
    System.out.println(num/g + "/" + den/g);
}
// addFractions(1,2,1,3) -> 5/6
```

2.4.10 Replace 0s with 1s in an Integer

```
public static int replaceZeros(int n) {
    return Integer.parseInt(Integer.toString(n).replace('0','1'));
}
// replaceZeros(102030) -> 112131
```

2.4.11 Number Expressible as Sum of Two Primes

```
public static boolean sumOfTwoPrimes(int n) {
    for (int i=2; i<=n/2; i++)
        if (isPrime(i) && isPrime(n-i)) {
            System.out.println(n+"="+i+"+"+(n-i));
            return true;
        }
    return false;
}
// sumOfTwoPrimes(34) -> 34=3+31  true
```

2.4.12 Quadratic Equation Roots

```
// ax^2 + bx + c = 0   D = b^2 - 4ac
public static void quadRoots(double a, double b, double c) {
    double D = b*b - 4*a*c;
    if (D > 0) {
        double r1 = (-b + Math.sqrt(D))/(2*a);
        double r2 = (-b - Math.sqrt(D))/(2*a);
        System.out.printf("Roots: %.4f and %.4f%n",r1,r2);
    } else if (D == 0) {
        System.out.printf("Equal root: %.4f%n", -b/(2*a));
    } else {
        double re = -b/(2*a), im = Math.sqrt(-D)/(2*a);
        System.out.printf("Complex: %.4f ± %.4fi%n",re,im);
    }
}
```

2.4.13 Sum of Numbers in a Range

```
// Sum 1+2+...+n = n*(n+1)/2  [Gauss formula]
long sumN(int n)         { return (long)n*(n+1)/2; }
long sumRange(int a, int b) { return sumN(b) - sumN(a-1); }
// sumRange(5,10) = 5+6+7+8+9+10 = 45
```

⚠ OVERFLOW IN NUMBER PROBLEMS

int: max ~2.1 billion (2^31-1). If n can be 10^6 and value 10^6, product overflows int.
Always cast to long before multiplying: (long)a * b
For factorials, 13! > int max, 21! > long max. Use BigInteger for n>=21.
Math.pow returns double — has precision limits. Use integer exponentiation for exact values.

🔍 WORKED EXAMPLES — Sample Questions & Answers

Q1: Is 40585 a strong number?

Digits: 4,0,5,8,5. Sum of factorials: 4!+0!+5!+8!+5! = 24+1+120+40320+120 = 40585. YES!

Q2: Find GCD and LCM of 56 and 98.

GCD: 98=56*1+42; 56=42*1+14; 42=14*3+0 -> GCD=14. LCM=56/14*98=392.

Q3: Are 76 and 625 automorphic?

76^2=5776 (ends in 76: YES). 625^2=390625 (ends in 625: YES).

Q4: Is 100 a Harshad number?

Digit sum=1+0+0=1. 100%1=0. YES! Is 19? Sum=10. 19%10=9. NO.

✏ PRACTICE QUESTIONS — Try These Yourself

8. Find all prime numbers between 50 and 100 using the Sieve of Eratosthenes.
9. Write code to check if 9474 is an Armstrong number (4 digits: $9^4+4^4+7^4+4^4$).
10. What is P(7,3)? Compute using the formula.
11. Compute sum of AP series with a=5, d=3, n=8. Show working.
12. Write code to print all perfect numbers below 1000.
13. Is the number 8128 perfect? Verify by finding all its divisors.
14. Reverse the digits of -9876. What is the result?

✔ SOLUTIONS TO PRACTICE QUESTIONS

S1: Primes 50-100: 53,59,61,67,71,73,79,83,89,97

S2: $9^4+4^4+7^4+4^4$ = 6561+256+2401+256 = 9474. YES, Armstrong!

S3: P(7,3) = 7*6*5 = 210

S4: Sum = 8/2*(10+21) = 4*31 = 124 [5,8,11,14,17,20,23,26]

S5: Perfect numbers below 1000: 6, 28, 496

S6: Divisors of 8128: 1,2,4,8,16,32,64,127,254,508,1016,2032,4064. Sum=8128. YES!

S7: reverseDigits(-9876) = -6789

Chapter 3: Number Systems

Number system conversions appear as complete TCS NQT problems. The exam may ask you to convert between any two bases, write a function to convert numbers to English words, or implement base conversions without using built-in helpers. This chapter covers the theory and code for all tested conversions.

3.1 The Four Number Systems

Base	Name	Digits Used	Example
2	Binary	0, 1	$1010_2 = 10_{10}$
8	Octal	0–7	$12_8 = 10_{10}$
10	Decimal	0–9	10_{10}
16	Hexadecimal	0–9, A–F	$A_{16} = 10_{10}$

JAVA BUILT-INS FOR NUMBER SYSTEMS

```
// Any base to decimal:
int n = Integer.parseInt("1010", 2);   // 10
int n = Integer.parseInt("FF",   16);  // 255
int n = Integer.parseInt("17",    8);  // 15

// Decimal to any base:
String b = Integer.toBinaryString(10);  // "1010"
String o = Integer.toOctalString(8);    // "10"
String h = Integer.toHexString(255);    // "ff"
String s = Integer.toString(10, 2);     // "1010"  (general)
```

3.2 Decimal ↔ Binary

3.2.1 Decimal to Binary — Divide by 2

Repeatedly divide by 2. Remainders collected in reverse give the binary representation.

```
public static String decToBin(int n) {
    if (n == 0) return "0";
    StringBuilder sb = new StringBuilder();
    while (n > 0) {
        sb.append(n % 2);   // collect remainders
        n /= 2;
    }
    return sb.reverse().toString();  // reverse to get MSB first
}
// decToBin(10) -> "1010"
```

```
// decToBin(255) -> "11111111"
```

3.2.2 Binary to Decimal — Positional Value

Each bit contributes bit × 2^position (counting from right, starting at 0). Or: accumulate left to right as result = result*2 + bit.

```
public static int binToDec(String bin) {
    int result = 0;
    for (int i = 0; i < bin.length(); i++) {
        result = result * 2 + (bin.charAt(i) - '0');
    }
    return result;
}
// binToDec("1010") -> 0*2+1=1, 1*2+0=2, 2*2+1=5, 5*2+0=10 -> 10

// Built-in: Integer.parseInt("1010", 2) -> 10
```

3.3 Decimal ↔ Octal

```
// Decimal -> Octal: divide by 8
public static String decToOct(int n) {
    if (n == 0) return "0";
    StringBuilder sb = new StringBuilder();
    while (n > 0) { sb.append(n % 8); n /= 8; }
    return sb.reverse().toString();
}
// decToOct(64) -> "100"   decToOct(255) -> "377"

// Octal -> Decimal: positional value base 8
public static int octToDec(String oct) {
    int result = 0;
    for (char c : oct.toCharArray())
        result = result * 8 + (c - '0');
    return result;
}
// octToDec("377") -> 255
```

3.4 Binary ↔ Octal

Key shortcut: Every 3 binary digits = 1 octal digit. Group from right, pad left with zeros.

```
// Binary -> Octal: group bits in 3 from the right
public static String binToOct(String bin) {
    while (bin.length() % 3 != 0) bin = "0" + bin;  // left-pad
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < bin.length(); i += 3) {
```

```
        String group = bin.substring(i, i+3);
        sb.append(Integer.parseInt(group, 2));  // 3 bits -> 1 octal digit
    }
    return sb.toString();
}
// binToOct("110101") -> groups: 110,101 -> 6,5 -> "65"

// Octal -> Binary: each octal digit -> 3 binary bits
public static String octToBin(String oct) {
    StringBuilder sb = new StringBuilder();
    for (char c : oct.toCharArray()) {
        String b = Integer.toBinaryString(c - '0');
        while (b.length() < 3) b = "0" + b;  // pad to 3 bits
        sb.append(b);
    }
    // Remove leading zeros
    String result = sb.toString().replaceFirst("^0+", "");
    return result.isEmpty() ? "0" : result;
}
// octToBin("65") -> 6->110, 5->101 -> "110101"
```

CONVERSION SHORTCUT TABLE

Binary -> Octal: Group 3 bits (right to left)
Binary -> Hex: Group 4 bits (right to left)
Octal -> Binary: Expand each digit to 3 bits
Hex -> Binary: Expand each digit to 4 bits
Any base -> Decimal: Use positional value (sum digit * base^position)
Decimal -> Any base: Repeated division, collect remainders in reverse

3.5 Numbers to Words

Converting digits/numbers to English words is a classic recursive decomposition problem. The key insight is to handle groups of three digits (thousands, millions) separately.

```
static final String[] ONES = {"","One","Two","Three","Four","Five",
    "Six","Seven","Eight","Nine","Ten","Eleven","Twelve",
    "Thirteen","Fourteen","Fifteen","Sixteen","Seventeen",
    "Eighteen","Nineteen"};
static final String[] TENS = {"","","Twenty","Thirty","Forty",
    "Fifty","Sixty","Seventy","Eighty","Ninety"};

public static String convert(int n) {
    if (n == 0) return "Zero";
    if (n < 0)  return "Minus " + convert(-n);
    if (n < 20) return ONES[n];
    if (n < 100) return TENS[n/10] + (n%10!=0 ? " "+ONES[n%10] : "");
    if (n < 1000) return ONES[n/100]+" Hundred"+(n%100!=0?" "+convert(n%100):"");
```

```
    if (n < 1_000_000) return convert(n/1000)+" Thousand"+(n%1000!=0?"
"+convert(n%1000):"");
    return convert(n/1_000_000)+" Million"+(n%1_000_000!=0?"
"+convert(n%1_000_000):"");
}
// convert(19)   -> "Nineteen"
// convert(100)  -> "One Hundred"
// convert(1234) -> "One Thousand Two Hundred Thirty Four"
// convert(1000001) -> "One Million One"
```

🔍 WORKED EXAMPLES — Sample Questions & Answers

Q1: Convert 255 to binary, octal, and hexadecimal.

Binary: 255÷2: rem 1,1,1,1,1,1,1,1 -> 11111111. Octal: decToOct(255) -> 377. Hex: toHexString(255) -> ff.

Q2: Convert binary 10110111 to octal.

Pad to 9 digits: 010 110 111. Groups: 010=2, 110=6, 111=7. Answer: 267.

Q3: Convert the number 1024 to words.

1024 = 1*1000 + 24. convert(1)*Thousand + convert(24). = One Thousand Twenty Four.

✏ PRACTICE QUESTIONS — Try These Yourself

15. Convert decimal 500 to binary. Show each division step.
16. Convert binary 11001100 to octal and decimal.
17. Write the number 99999 in words.
18. Convert octal 777 to decimal and binary.
19. What is the hex representation of 4096?

✔ SOLUTIONS TO PRACTICE QUESTIONS

S1: 500÷2: 0,0,1,0,1,1,1,1,1 -> binary: 111110100

S2: 11001100: pad->011 001 100. Groups: 3,1,4 -> Octal:314. Decimal: 128+64+8+4=204.

S3: Ninety Nine Thousand Nine Hundred Ninety Nine

S4: 777 octal: 7*64+7*8+7=448+56+7=511 decimal. Binary: 111 111 111 = 111111111.

S5: Integer.toHexString(4096) = "1000"

Chapter 4: Sorting Algorithms

Sorting is one of the most fundamental operations in computing. Understanding how sorting algorithms work — not just using Arrays.sort() — is essential for TCS NQT because questions may explicitly ask you to implement a specific algorithm, or understanding the internal mechanism may be required for a problem's logic.

4.1 Complexity Overview

Algorithm	Best	Average	Worst	Space	Stable?
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)	No
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)	No
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes

WHEN TO USE WHICH SORT

Small array (n<20) or nearly sorted: Insertion Sort wins in practice.

General purpose, memory tight: Quick Sort (avg O(n log n), O(log n) space).

Stability required OR worst-case guarantee: Merge Sort.

Minimise writes (e.g. flash memory): Selection Sort (only O(n) swaps).

In exam: always use Arrays.sort() unless a specific algorithm is asked for.

4.2 Bubble Sort

Bubble Sort compares adjacent elements and swaps them if they are in the wrong order. After each full pass, the largest unsorted element 'bubbles' to its correct position at the end.

Optimization: If no swap occurs during a pass, the array is already sorted — terminate early. This gives O(n) best case for sorted input.

```
public static void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int pass = 0; pass < n-1; pass++) {
        boolean swapped = false;
        for (int j = 0; j < n-1-pass; j++) {  // last 'pass' elements are sorted
            if (arr[j] > arr[j+1]) {
                int tmp = arr[j]; arr[j]=arr[j+1]; arr[j+1]=tmp;
                swapped = true;
            }
        }
        if (!swapped) break;   // early exit if already sorted
    }
}
```

```
// TRACE on {64,34,25,12,22,11,90}:
// Pass 1: 34 25 12 22 11 64 90  (64 bubbled past 34,25...)
// Pass 2: 25 12 22 11 34 64 90
// Pass 3: 12 22 11 25 34 64 90
// ... until sorted: 11 12 22 25 34 64 90
```

4.3 Selection Sort

Selection Sort divides the array into a sorted left portion and an unsorted right portion. In each pass, it finds the minimum element in the unsorted portion and swaps it to the front of the unsorted portion.

Key property: Makes at most O(n) swaps — useful when writes are expensive.

```
public static void selectionSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n-1; i++) {
        int minIdx = i;
        for (int j = i+1; j < n; j++)
            if (arr[j] < arr[minIdx]) minIdx = j;
        // Swap minimum to position i
        int tmp = arr[i]; arr[i]=arr[minIdx]; arr[minIdx]=tmp;
    }
}

// TRACE on {5,3,1,4,2}:
// i=0: minIdx=2(val 1), swap: {1,3,5,4,2}
// i=1: minIdx=4(val 2), swap: {1,2,5,4,3}
// i=2: minIdx=4(val 3), swap: {1,2,3,4,5}  <- done!
```

4.4 Insertion Sort

Insertion Sort builds a sorted sub-array one element at a time. It picks the next unsorted element (the 'key') and shifts larger sorted elements right to make room, then inserts the key.

Think of how you sort playing cards in your hand — you pick one card and slide it to its correct position among the already-sorted cards.

```
public static void insertionSort(int[] arr) {
    int n = arr.length;
    for (int i = 1; i < n; i++) {
        int key = arr[i];         // element to insert
        int j   = i - 1;
        // Shift elements greater than key one position right
        while (j >= 0 && arr[j] > key) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;           // insert key in correct position
```

```
    }
}

// TRACE on {5,1,4,2}:
// i=1: key=1, shift 5 right -> {5,5,4,2}, insert: {1,5,4,2}
// i=2: key=4, shift 5 right -> {1,5,5,2}, insert: {1,4,5,2}
// i=3: key=2, shift 5,4 right, insert: {1,2,4,5}
```

4.5 Quick Sort

Quick Sort is a divide-and-conquer algorithm. It selects a 'pivot' element, partitions the array into elements ≤ pivot (left) and > pivot (right), then recursively sorts both sides.

Partition (Lomuto scheme): Pivot = last element. Use an index i to mark the boundary of elements ≤ pivot. Scan with j; when arr[j] ≤ pivot, increment i and swap arr[i] with arr[j].

```
private static int partition(int[] arr, int low, int high) {
    int pivot = arr[high];    // last element as pivot
    int i = low - 1;          // boundary of elements <= pivot

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            int tmp = arr[i]; arr[i]=arr[j]; arr[j]=tmp;
        }
    }
    // Place pivot in its final correct position
    int tmp = arr[i+1]; arr[i+1]=arr[high]; arr[high]=tmp;
    return i + 1;  // pivot index
}

public static void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi-1);    // sort left of pivot
        quickSort(arr, pi+1, high);   // sort right of pivot
    }
}
// Call: quickSort(arr, 0, arr.length-1)

// TRACE on {10,7,8,9,1,5}, pivot=5:
// After partition: {1,5,8,9,7,10}  pi=1
// Left: {1} already sorted. Right: quickSort({8,9,7,10})
```

⚠ QUICK SORT WORST CASE

Worst case O(n²) occurs when pivot is always the min or max (e.g. sorted array).

FIX: Use random pivot -> swap arr[random(low,high)] with arr[high] before partitioning.

Random pivot brings expected case to O(n log n) with very high probability.

4.6 Merge Sort

Merge Sort is a stable divide-and-conquer algorithm. It recursively splits the array in half until subarrays have 1 element (trivially sorted), then merges sorted pairs back together.

Merge step: Compare front elements of both halves, always taking the smaller one into the output. When one half is exhausted, copy the rest of the other.

```
private static void merge(int[] arr, int l, int m, int r) {
    // Create copies of both halves
    int[] L = Arrays.copyOfRange(arr, l, m+1);
    int[] R = Arrays.copyOfRange(arr, m+1, r+1);
    int i=0, j=0, k=l;
    // Merge back into arr[l..r]
    while (i < L.length && j < R.length) {
        if (L[i] <= R[j]) arr[k++] = L[i++];  // <= keeps stability
        else              arr[k++] = R[j++];
    }
    while (i < L.length) arr[k++] = L[i++];
    while (j < R.length) arr[k++] = R[j++];
}

public static void mergeSort(int[] arr, int l, int r) {
    if (l < r) {
        int m = l + (r-l)/2;    // midpoint (overflow-safe)
        mergeSort(arr, l, m);
        mergeSort(arr, m+1, r);
        merge(arr, l, m, r);
    }
}
// Call: mergeSort(arr, 0, arr.length-1)

// TRACE on {38,27,43,3,9,82,10}:
// Split: {38,27,43,3} | {9,82,10}
// Merge pairs, then halves: -> {3,9,10,27,38,43,82}
```

4.7 Java's Built-in Sorting

```
import java.util.*;

// Primitives: uses Dual-Pivot QuickSort (avg O(n log n))
int[] arr = {5,3,1,4,2};
Arrays.sort(arr);                       // [1,2,3,4,5]
Arrays.sort(arr, 1, 4);                 // sort only indices 1,2,3

// Object arrays / Collections: uses TimSort (stable, O(n log n))
```

```
Integer[] boxed = {5,3,1,4,2};
Arrays.sort(boxed, Collections.reverseOrder()); // [5,4,3,2,1]

List<Integer> list = Arrays.asList(5,3,1,4,2);
Collections.sort(list);                  // [1,2,3,4,5]
Collections.sort(list, (a,b)->b-a);      // [5,4,3,2,1] descending

// Sort String array alphabetically:
String[] words = {"banana","apple","cherry"};
Arrays.sort(words);                      // [apple,banana,cherry]
```

🔍 WORKED EXAMPLES — Sample Questions & Answers

Q1: Trace Bubble Sort on {5,1,4,2,8}. Show after each pass.

Pass 1: compare 5>1->swap, 5>4->swap, 5>2->swap, 5<8->no. -> {1,4,2,5,8}

Pass 2: 1<4->no, 4>2->swap, 4<5->no. -> {1,2,4,5,8}

Pass 3: no swaps -> exit early. Final: {1,2,4,5,8}

Q2: How many comparisons does Selection Sort make on an array of n=5?

Pass 0: 4 comparisons. Pass 1: 3. Pass 2: 2. Pass 3: 1. Total: 4+3+2+1 = 10 = n(n-1)/2.

Q3: Why does Merge Sort always run in O(n log n) while Quick Sort doesn't?

Merge Sort always splits exactly in half (log n levels × O(n) work per level = O(n log n)). Quick Sort split depends on pivot quality — a bad pivot (always min/max) gives O(n) levels -> O(n²).

✏ PRACTICE QUESTIONS — Try These Yourself

20. Sort {3,6,8,10,1,2,1} using Insertion Sort. Show the array after inserting each element.
21. Implement Quick Sort and trace it on {4,3,2,1}. Identify the pivot at each step.
22. What is the minimum number of swaps to sort {2,3,4,1,5} using Selection Sort?
23. Merge Sort {6,5,4,3,2,1}. Draw the split and merge tree.
24. Write code to sort an Integer[] in descending order using Collections API.

✔ SOLUTIONS TO PRACTICE QUESTIONS

S1: Insertion Sort trace on {3,6,8,10,1,2,1}:

```
i=1: key=6, 6>3 -> no shift: {3,6,8,10,1,2,1}
i=2: key=8, 8>6 -> no shift: {3,6,8,10,1,2,1}
i=3: key=10 -> no shift: {3,6,8,10,1,2,1}
i=4: key=1, shift 10,8,6,3 right: {1,3,6,8,10,2,1}
i=5: key=2, shift 10,8,6,3 right: {1,2,3,6,8,10,1}
i=6: key=1, shift 10,8,6,3,2 right: {1,1,2,3,6,8,10}
```

S3: Swaps in Selection Sort on {2,3,4,1,5}:

Pass 0: min=1 at idx 3, swap with idx 0: {1,3,4,2,5}. Pass 1: min=2 at idx 3, swap with idx 1: {1,2,4,3,5}. Pass 2: min=3 at idx 3, swap with idx 2: {1,2,3,4,5}. Total swaps: 3.

S5: Integer[] arr2 = {5,3,1,4,2}; Arrays.sort(arr2, Collections.reverseOrder());

Chapter 5: Strings

Strings are sequences of characters. In Java, String objects are immutable — every modification creates a new object. This has critical performance implications: never concatenate inside a loop using '+'. Always use StringBuilder for mutable string building.

5.1 String Fundamentals in Java

```
String s = "Hello, World!";

// ── CORE METHODS ──────────────────────────────────────────
s.length()              // 13
s.charAt(0)             // 'H'
s.indexOf('o')          // 4  (first occurrence)
s.lastIndexOf('o')      // 8  (last occurrence)
s.substring(7)          // "World!"
s.substring(7, 12)      // "World" (end index EXCLUSIVE)
s.toLowerCase()         // "hello, world!"
s.toUpperCase()         // "HELLO, WORLD!"
s.trim()                // remove leading/trailing whitespace
s.strip()               // Java 11+, also removes Unicode spaces
s.replace('l', 'x')     // "Hexxo, Worxd!"
s.replace("World","Java")// "Hello, Java!"
s.replaceAll("[aeiou]","")// remove all vowels (regex)
s.contains("World")     // true
s.startsWith("Hello")   // true
s.endsWith("!")         // true
s.isEmpty()             // false
s.isBlank()             // false (Java 11+)
s.split(",\\s*")        // ["Hello", "World!"]
s.toCharArray()          // char[]
s.equals("other")       // true/false — ALWAYS use equals(), not ==
s.equalsIgnoreCase("HELLO, WORLD!") // true
s.compareTo("other")    // lexicographic comparison (negative/0/positive)
String.valueOf(42)       // "42"  (int to String)
Integer.parseInt("42")  // 42   (String to int)
```

```
// ── StringBuilder (MUTABLE — use in loops) ──────────────
StringBuilder sb = new StringBuilder();
sb.append('H');          // H
sb.append("ello");       // Hello
sb.insert(5, '!');       // Hello!
sb.deleteCharAt(5);      // Hello
sb.delete(1, 3);         // Hlo
sb.reverse();            // olH
sb.replace(0,2,"XX");    // XXH
sb.length();             // current length
```

```
sb.charAt(0);            // 'X'
sb.setCharAt(0,'A');     // modify in place
sb.toString();           // convert to String
```

⚠ STRING vs STRINGBUILDER PERFORMANCE

String s = "";
for(int i=0;i<1000;i++) s += 'a'; // O(n²) — creates 1000 new String objects!

StringBuilder sb = new StringBuilder();
for(int i=0;i<1000;i++) sb.append('a'); // O(n) — single object
String s = sb.toString();

Rule: Any loop that builds a string must use StringBuilder.

🔗 ASCII QUICK REFERENCE

'A'=65, 'Z'=90 ('a'=97, 'z'=122) difference=32
'0'=48, '9'=57
char to int: int x = c - '0'; (for digit chars '0'..'9')
lowercase: char lc = (char)(c + 32); (only if c is uppercase)
uppercase: char uc = (char)(c - 32); (only if c is lowercase)
Check alpha: Character.isLetter(c)
Check digit: Character.isDigit(c)
Check alpha-num: Character.isLetterOrDigit(c)

5.2 String Algorithms

5.2.1 Palindrome Check

```
// Method 1: Two-pointer (O(n) time, O(1) space)
public static boolean isPalindrome(String s) {
    int l=0, r=s.length()-1;
    while (l < r) {
        if (s.charAt(l) != s.charAt(r)) return false;
        l++; r--;
    }
    return true;
}

// Method 2: Reverse and compare
public static boolean isPalindrome2(String s) {
    return s.equals(new StringBuilder(s).reverse().toString());
}
// isPalindrome("racecar") -> true
// Ignore spaces/case: s.toLowerCase().replaceAll("[^a-z0-9]","")
```

5.2.2 Count Vowels, Consonants, Spaces

```
public static void countVCS(String s) {
    int v=0, con=0, sp=0;
    for (char c : s.toLowerCase().toCharArray()) {
        if (c==' ')                     sp++;
        else if ("aeiou".indexOf(c)>=0)  v++;
        else if (c>='a' && c<='z')       con++;
    }
    System.out.printf("Vowels:%d Consonants:%d Spaces:%d%n",v,con,sp);
}
// countVCS("Hello World") -> Vowels:3 Consonants:7 Spaces:1
```

5.2.3 Remove Characters (Vowels / Spaces / Non-Alpha)

```
// Remove vowels
String noVowels = s.replaceAll("[aeiouAEIOU]", "");

// Remove all spaces
String noSpaces = s.replace(" ", "");
// or: s.replaceAll("\\s+", "");  // removes all whitespace types

// Remove non-alphabetic characters
String alphaOnly = s.replaceAll("[^a-zA-Z]", "");

// Remove brackets (keep content)
String noBrackets = s.replaceAll("[()\\[\\]{}]", "");
// noBrackets("(a+b)*[c-{d/e}]") -> "a+b*cd/e"

// Remove characters present in another string
public static String removeCharsIn(String s, String remove) {
    Set<Character> toRemove = new HashSet<>();
    for (char c : remove.toCharArray()) toRemove.add(c);
    StringBuilder sb = new StringBuilder();
    for (char c : s.toCharArray())
        if (!toRemove.contains(c)) sb.append(c);
    return sb.toString();
}
```

5.2.4 Reverse a String / Reverse Words

```
// Reverse entire string
public static String reverseString(String s) {
    return new StringBuilder(s).reverse().toString();
}

// Reverse words
public static String reverseWords(String s) {
```

```
    String[] words = s.trim().split("\\s+");
    StringBuilder sb = new StringBuilder();
    for (int i=words.length-1; i>=0; i--) {
        sb.append(words[i]);
        if (i>0) sb.append(' ');
    }
    return sb.toString();
}
// reverseWords("Hello World Java") -> "Java World Hello"
```

5.2.5 Character Frequency & Max Occurring Character

```
// Frequency using int array (fastest for ASCII)
public static int[] charFreq(String s) {
    int[] freq = new int[256];
    for (char c : s.toCharArray()) freq[c]++;
    return freq;
}

// Max occurring character
public static char maxOccurring(String s) {
    int[] freq = charFreq(s);
    int maxF=0; char result='\0';
    for (int i=0; i<256; i++)
        if (freq[i]>maxF) { maxF=freq[i]; result=(char)i; }
    return result;
}
// maxOccurring("aabbbcccc") -> 'c'

// Non-repeating characters (in order)
public static void nonRepeating(String s) {
    int[] freq = charFreq(s);
    for (char c : s.toCharArray())
        if (freq[c]==1) System.out.print(c+" ");
}
// nonRepeating("programming") -> p o a i
```

5.2.6 Remove / Print Duplicates

```
// Remove all duplicates (keep first occurrence)
public static String removeDuplicates(String s) {
    boolean[] seen = new boolean[256];
    StringBuilder sb = new StringBuilder();
    for (char c : s.toCharArray())
        if (!seen[c]) { sb.append(c); seen[c]=true; }
    return sb.toString();
}
// removeDuplicates("programming") -> "progamin"
```

```
// Print duplicate characters
public static void printDuplicates(String s) {
    int[] freq = new int[256];
    for (char c : s.toCharArray()) freq[c]++;
    for (int i=0; i<256; i++)
        if (freq[i]>1) System.out.print((char)i+"("+freq[i]+") ");
}
// printDuplicates("programming") -> g(2) m(2) r(2)
```

5.2.7 Anagram Check

```
// Method 1: Sort-based O(n log n)
public static boolean isAnagram(String a, String b) {
    char[] ca = a.toLowerCase().toCharArray();
    char[] cb = b.toLowerCase().toCharArray();
    Arrays.sort(ca); Arrays.sort(cb);
    return Arrays.equals(ca, cb);
}

// Method 2: Frequency-based O(n)
public static boolean isAnagram2(String a, String b) {
    if (a.length()!=b.length()) return false;
    int[] count = new int[256];
    for (char c : a.toCharArray()) count[c]++;
    for (char c : b.toCharArray()) { count[c]--; if(count[c]<0) return false; }
    return true;
}
// isAnagram("listen","silent") -> true
```

5.2.8 Wildcard String Matching

```
// '?' matches any single char, '*' matches any sequence (including empty)
public static boolean wildcard(String text, String pat) {
    int t=text.length(), p=pat.length();
    boolean[][] dp = new boolean[t+1][p+1];
    dp[0][0] = true;
    for (int j=1;j<=p;j++) if(pat.charAt(j-1)=='*') dp[0][j]=dp[0][j-1];
    for (int i=1;i<=t;i++) {
        for (int j=1;j<=p;j++) {
            char pc=pat.charAt(j-1);
            if (pc=='*')                          dp[i][j]=dp[i-1][j]||dp[i][j-
1];
            else if (pc=='?'||pc==text.charAt(i-1)) dp[i][j]=dp[i-1][j-1];
        }
    }
    return dp[t][p];
}
```

5.2.9 Sum of Numbers in a String

```
public static long sumOfNumbers(String s) {
    long sum=0; int i=0;
    while (i < s.length()) {
        if (Character.isDigit(s.charAt(i))) {
            long num=0;
            while (i<s.length() && Character.isDigit(s.charAt(i)))
                num = num*10 + (s.charAt(i++)-'0');
            sum += num;
        } else i++;
    }
    return sum;
}
// sumOfNumbers("abc12def34") -> 46
```

5.2.10 Capitalize First and Last Character of Each Word

```
public static String capitalizeFirstLast(String s) {
    StringBuilder result = new StringBuilder();
    for (String word : s.split(" ")) {
        if (word.length()==0) continue;
        if (word.length()==1) {
            result.append(Character.toUpperCase(word.charAt(0)));
        } else {
            result.append(Character.toUpperCase(word.charAt(0)));
            result.append(word, 1, word.length()-1);
            result.append(Character.toUpperCase(word.charAt(word.length()-1)));
        }
        result.append(' ');
    }
    return result.toString().trim();
}
// "hello world" -> "HellO WorlD"
```

5.2.11 Largest Word in a String

```
public static String largestWord(String s) {
    String[] words = s.trim().split("\\s+");
    String largest = "";
    for (String w : words) if (w.length()>largest.length()) largest=w;
    return largest;
}
// largestWord("I love programming") -> "programming"
```

5.2.12 Sort Characters in a String

```
public static String sortChars(String s) {
    char[] c = s.toCharArray();
    Arrays.sort(c);
    return new String(c);
}
// sortChars("programming") -> "aggimmnoprr"
```

5.2.13 Count Words in a String

```
public static int countWords(String s) {
    s = s.trim();
    if (s.isEmpty()) return 0;
    return s.split("\\s+").length;
}
// countWords("  Hello   World  ") -> 2
```

5.2.14 Word with Most Repeated Letters

```
public static String wordMostRepeats(String s) {
    String best=""; int bestCount=0;
    for (String w : s.trim().split("\\s+")) {
        int[] freq = new int[256];
        for (char c : w.toCharArray()) freq[c]++;
        int reps=0;
        for (int f : freq) if (f>1) reps+=f;
        if (reps>bestCount) { bestCount=reps; best=w; }
    }
    return best;
}
```

5.2.15 Change Case / Concatenate / Find Substring

```
// Toggle case of each character
public static String toggleCase(String s) {
    StringBuilder sb = new StringBuilder();
    for (char c : s.toCharArray()) {
        if (Character.isUpperCase(c)) sb.append(Character.toLowerCase(c));
        else if (Character.isLowerCase(c)) sb.append(Character.toUpperCase(c));
        else sb.append(c);
    }
    return sb.toString();
}
// toggleCase("Hello World") -> "hELLO wORLD"

// Concatenate two strings
String result = s1.concat(s2);   // or s1 + s2  or sb.append(s2)
```

```
// Find substring and its starting position
int pos = text.indexOf(pattern);  // -1 if not found
if (pos >= 0) System.out.println("Found at index: " + pos);
```

5.2.16 Next Lexicographic Character

```
public static String nextLex(String s) {
    StringBuilder sb = new StringBuilder();
    for (char c : s.toCharArray()) {
        if (c=='z') sb.append('a');
        else if (c=='Z') sb.append('A');
        else if (Character.isLetter(c)) sb.append((char)(c+1));
        else sb.append(c);
    }
    return sb.toString();
}
// nextLex("hello") -> "ifmmp"  nextLex("xyz") -> "yza"
```

5.2.17 Common Subsequences (LCS Length)

```
public static int lcsLength(String s1, String s2) {
    int m=s1.length(), n=s2.length();
    int[][] dp = new int[m+1][n+1];
    for (int i=1;i<=m;i++)
        for (int j=1;j<=n;j++)
            dp[i][j] = s1.charAt(i-1)==s2.charAt(j-1) ?
                       1+dp[i-1][j-1] : Math.max(dp[i-1][j],dp[i][j-1]);
    return dp[m][n];
}
// lcsLength("ABCBDAB","BDCAB") -> 4  (BCAB or BDAB)
```

🔍 WORKED EXAMPLES — Sample Questions & Answers

Q1: Count frequency of each char in "mississippi".

m:1, i:4, s:4, p:2. Using int[256] array: freq['m']++, etc.

Q2: Are "Astronomer" and "Moon starer" anagrams (ignore spaces)?

Remove spaces and lowercase both: "astronomer" vs "moonstarer". Both sorted: "aemnoorrst". YES, anagram!

Q3: Reverse words in "The quick brown fox".

Split: ["The","quick","brown","fox"]. Reverse iterate: "fox brown quick The".

✏ PRACTICE QUESTIONS — Try These Yourself

25. Check if "Madam Im Adam" is a palindrome (ignore spaces and case).
26. Find the largest word in: "Java is a powerful programming language".
27. Remove duplicates from "banana". What is the result?

28. Apply nextLex() to "xyz". What do you get?
29. Compute LCS length of "AGGTAB" and "GXTXAYB".
30. Find the character that occurs most in "abracadabra".
31. Sum of numbers in "I have 2 cats and 3 dogs with 10 paws".

✔ SOLUTIONS TO PRACTICE QUESTIONS

S1: Remove spaces, lowercase: "madamimadam". Reverse = same. YES palindrome.
S2: "programming" (11 chars) is the longest word.
S3: removeDuplicates("banana"): b(add),a(add),n(add),a(skip),n(skip),a(skip) -> "ban"
S4: nextLex("xyz"): x->y, y->z, z->a -> "yza"
S5: LCS("AGGTAB","GXTXAYB") = 4 (GTAB)
S6: maxOccurring("abracadabra"): freq: a=5,b=2,r=2,c=1,d=1 -> 'a'
S7: sumOfNumbers("I have 2 cats and 3 dogs with 10 paws") = 2+3+10 = 15

Chapter 6: Matrices (2D Arrays)

A matrix is a two-dimensional array — a grid of rows and columns. Matrices are used in image processing, graph adjacency, dynamic programming, and game boards. TCS NQT problems typically involve traversal, transformation (transpose, rotation), searching, and mathematical operations on matrices.

6.1 Matrix Basics in Java

```
// Declaration and initialisation
int[][] mat = new int[3][4];        // 3 rows, 4 columns, filled with 0
int[][] mat = {{1,2,3},{4,5,6},{7,8,9}}; // 3x3 literal

// Dimensions
int rows = mat.length;              // number of rows
int cols = mat[0].length;           // number of columns

// Access
mat[i][j] = 42;                     // set element at row i, column j
int x = mat[i][j];                  // get element

// Row-by-row traversal (ROW MAJOR — cache-friendly)
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        System.out.print(mat[i][j] + "\t");
    }
    System.out.println();
}

// Reading a matrix from input
Scanner sc = new Scanner(System.in);
int r = sc.nextInt(), c = sc.nextInt();
int[][] m = new int[r][c];
for (int i=0; i<r; i++)
    for (int j=0; j<c; j++)
        m[i][j] = sc.nextInt();
```

6.2 Matrix Traversal Patterns

6.2.1 Diagonal Traversal

```
// Main diagonal (top-left to bottom-right): i == j
for (int i=0; i<n; i++) System.out.print(mat[i][i] + " ");

// Anti-diagonal (top-right to bottom-left): i + j == n-1
```

```
for (int i=0; i<n; i++) System.out.print(mat[i][n-1-i] + " ");

// Sum of diagonals (for square matrix)
int mainSum=0, antiSum=0;
for (int i=0; i<n; i++) {
    mainSum += mat[i][i];
    antiSum += mat[i][n-1-i];
}
// Avoid double-counting centre for odd n:
if (n%2!=0) antiSum -= mat[n/2][n/2]; // if you're combining both diagonals
```

6.2.2 Spiral Order Traversal

```
public static List<Integer> spiralOrder(int[][] mat) {
    List<Integer> result = new ArrayList<>();
    int top=0, bottom=mat.length-1, left=0, right=mat[0].length-1;

    while (top<=bottom && left<=right) {
        // Right across top row
        for (int j=left; j<=right; j++) result.add(mat[top][j]);
        top++;
        // Down right column
        for (int i=top; i<=bottom; i++) result.add(mat[i][right]);
        right--;
        // Left across bottom row
        if (top<=bottom) {
            for (int j=right; j>=left; j--) result.add(mat[bottom][j]);
            bottom--;
        }
        // Up left column
        if (left<=right) {
            for (int i=bottom; i>=top; i--) result.add(mat[i][left]);
            left++;
        }
    }
    return result;
}
// {{1,2,3},{4,5,6},{7,8,9}} -> [1,2,3,6,9,8,7,4,5]
```

6.2.3 Row-wise and Column-wise Search

```
// Linear search (works for any matrix)
public static int[] search(int[][] mat, int target) {
    for (int i=0; i<mat.length; i++)
        for (int j=0; j<mat[0].length; j++)
            if (mat[i][j]==target) return new int[]{i,j};
    return new int[]{-1,-1};
}
```

```
// Optimised search in sorted matrix (each row & col sorted ascending)
// Start top-right corner: if >target go left, if <target go down
public static int[] searchSorted(int[][] mat, int target) {
    int r=0, c=mat[0].length-1;
    while (r<mat.length && c>=0) {
        if (mat[r][c]==target) return new int[]{r,c};
        else if (mat[r][c]>target) c--;
        else r++;
    }
    return new int[]{-1,-1};
}
```

6.3 Matrix Transformations

6.3.1 Transpose

Transpose swaps rows and columns: element [i][j] moves to [j][i]. For an n×n matrix, this can be done in-place by swapping (i,j) with (j,i) for j>i.

```
// In-place transpose (square matrix only)
public static void transpose(int[][] mat) {
    int n = mat.length;
    for (int i=0; i<n; i++)
        for (int j=i+1; j<n; j++) {
            int tmp=mat[i][j]; mat[i][j]=mat[j][i]; mat[j][i]=tmp;
        }
}
// {{1,2,3},{4,5,6},{7,8,9}} -> {{1,4,7},{2,5,8},{3,6,9}}

// For non-square m×n matrix (creates new n×m matrix)
public static int[][] transposeRect(int[][] mat) {
    int r=mat.length, c=mat[0].length;
    int[][] result = new int[c][r];
    for (int i=0; i<r; i++)
        for (int j=0; j<c; j++)
            result[j][i] = mat[i][j];
    return result;
}
```

6.3.2 Rotate 90° Clockwise

Clockwise 90° = Transpose + Reverse each row.

```
public static void rotate90CW(int[][] mat) {
    int n = mat.length;
    // Step 1: Transpose
    for (int i=0; i<n; i++)
```

```
        for (int j=i+1; j<n; j++) {
            int tmp=mat[i][j]; mat[i][j]=mat[j][i]; mat[j][i]=tmp;
        }
    // Step 2: Reverse each row
    for (int i=0; i<n; i++) {
        int l=0, r=n-1;
        while(l<r){ int t=mat[i][l];mat[i][l]=mat[i][r];mat[i][r]=t;l++;r--; }
    }
}

// Anti-clockwise 90° = Transpose + Reverse each COLUMN
// 180° = Rotate CW twice, or simply reverse each row then each column
```

6.3.3 Matrix Addition, Subtraction, Multiplication

```
// Addition: A + B (same dimensions required)
public static int[][] add(int[][] A, int[][] B) {
    int r=A.length, c=A[0].length;
    int[][] C = new int[r][c];
    for (int i=0;i<r;i++) for (int j=0;j<c;j++) C[i][j]=A[i][j]+B[i][j];
    return C;
}

// Multiplication: A (m×k) * B (k×n) = C (m×n)   O(m*k*n)
public static int[][] multiply(int[][] A, int[][] B) {
    int m=A.length, k=A[0].length, n=B[0].length;
    int[][] C = new int[m][n];
    for (int i=0;i<m;i++)
        for (int j=0;j<n;j++)
            for (int p=0;p<k;p++)
                C[i][j] += A[i][p]*B[p][j];
    return C;
}
```

6.3.4 Find Min / Max in Matrix

```
public static int[] matMinMax(int[][] mat) {
    int min=mat[0][0], max=mat[0][0];
    for (int[] row : mat)
        for (int x : row) {
            if (x<min) min=x;
            if (x>max) max=x;
        }
    return new int[]{min,max};
}
```

6.3.5 Row/Column Sum — Prefix Sum for 2D

```
// Row and column sums
for (int i=0; i<r; i++) {
    int rowSum=0;
    for (int j=0; j<c; j++) rowSum+=mat[i][j];
    System.out.println("Row "+i+" sum: "+rowSum);
}

// 2D prefix sum for range queries
int[][] pre = new int[r+1][c+1];
for (int i=1;i<=r;i++)
    for (int j=1;j<=c;j++)
        pre[i][j]=mat[i-1][j-1]+pre[i-1][j]+pre[i][j-1]-pre[i-1][j-1];

// Sum of sub-matrix from (r1,c1) to (r2,c2) (0-indexed):
int subSum = pre[r2+1][c2+1] - pre[r1][c2+1] - pre[r2+1][c1] + pre[r1][c1];
```

🔍 WORKED EXAMPLES — Sample Questions & Answers

Q1: Find the sum of main and anti diagonals of {{1,2,3},{4,5,6},{7,8,9}}.

Main diag: (0,0)+(1,1)+(2,2) = 1+5+9=15. Anti diag: (0,2)+(1,1)+(2,0) = 3+5+7=15.

Q2: Transpose {{1,2,3},{4,5,6},{7,8,9}}. Show the result.

After transpose: {{1,4,7},{2,5,8},{3,6,9}}. Row 0 becomes column 0, etc.

Q3: Rotate {{1,2,3},{4,5,6},{7,8,9}} 90° clockwise.

After transpose: {{1,4,7},{2,5,8},{3,6,9}}. Reverse rows: {{7,4,1},{8,5,2},{9,6,3}}.

✏ PRACTICE QUESTIONS — Try These Yourself

32. Write code to print a matrix in spiral order for input {{1,2,3,4},{5,6,7,8},{9,10,11,12}}.
33. Find the element at position (1,2) after rotating {{1,2,3},{4,5,6},{7,8,9}} 90° anti-clockwise.
34. Multiply matrices A={{1,2},{3,4}} and B={{5,6},{7,8}}. What is C[1][1]?
35. Search for element 7 in sorted matrix {{1,2,3},{4,5,6},{7,8,9}} using optimised top-right method. How many comparisons?
36. Compute the sum of the sub-matrix from (0,0) to (1,1) in {{1,2,3},{4,5,6},{7,8,9}} using 2D prefix sum.

✔ SOLUTIONS TO PRACTICE QUESTIONS

S1: Spiral: [1,2,3,4,8,12,11,10,9,5,6,7]

S2: Anti-CW = Transpose + reverse each column. Transpose: {{1,4,7},{2,5,8},{3,6,9}}. Reverse cols -> {{3,6,9},{2,5,8},{1,4,7}}. Element at (1,2) = 8.

S3: C[1][1] = 3*6+4*8 = 18+32 = 50.

S4: Start at (0,2)=3<7,go down.(1,2)=6<7,go down.(2,2)=9>7,go left.(2,1)=8>7,go left.(2,0)=7==target. 5 comparisons.

S5: pre[2][2] = 1+2+4+5=12. Using formula: pre[2][2]-pre[0][2]-pre[2][0]+pre[0][0] = 12-0-0+0 = 12.

Chapter 7: Collections Framework

Java's Collections Framework provides ready-made data structures and algorithms. Mastering the key classes — ArrayList, LinkedList, Stack, Queue, PriorityQueue, HashMap, HashSet, TreeMap — means you can solve complex TCS NQT problems in fewer lines and with fewer bugs.

7.1 The Collections Hierarchy

Interface / Class	Type	Key Property
ArrayList<E>	List	Dynamic array; O(1) get, O(n) insert
LinkedList<E>	List/Deque	Doubly-linked; O(1) head/tail ops
Stack<E>	LIFO	push/pop/peek — last in first out
ArrayDeque<E>	Deque/Queue	Preferred Queue impl — faster than LinkedList
PriorityQueue<E>	Min-Heap	poll() returns smallest element
HashMap<K,V>	Map	O(1) avg get/put; no order
LinkedHashMap<K,V>	Map	HashMap + insertion order
TreeMap<K,V>	Sorted Map	O(log n) get/put; keys sorted
HashSet<E>	Set	O(1) contains; no duplicates, no order
LinkedHashSet<E>	Set	HashSet + insertion order
TreeSet<E>	Sorted Set	O(log n); sorted unique elements

7.2 ArrayList

📖 **WHEN TO USE**

Use ArrayList when you need a resizable array with fast random access (O(1) by index).

```
import java.util.*;

ArrayList<Integer> list = new ArrayList<>();
list.add(10);               // [10]
list.add(20);               // [10, 20]
list.add(1, 15);            // [10, 15, 20]  (insert at index 1)
list.remove(1);             // [10, 20]       (remove by index)
list.remove(Integer.valueOf(10)); // [20]     (remove by value)
list.get(0);                // 20
list.set(0, 99);            // [99]  (update)
list.size();                // 1
list.contains(99);          // true
list.indexOf(99);           // 0
Collections.sort(list);     // sort ascending
Collections.reverse(list);  // reverse
Collections.shuffle(list);  // random shuffle
```

```
Collections.frequency(list, 99); // count occurrences
Collections.max(list);      // max element
Collections.min(list);      // min element

// Convert array to list:
Integer[] arr = {3,1,2};
List<Integer> l = new ArrayList<>(Arrays.asList(arr));

// Convert list to array:
Integer[] a = list.toArray(new Integer[0]);

// Iterate:
for (int x : list) System.out.print(x + " ");
list.forEach(System.out::println);  // Java 8+
```

7.3 Stack

📖 WHEN TO USE

LIFO structure. Classic uses: matching brackets, undo operations, DFS traversal, expression evaluation.

```
Stack<Integer> stack = new Stack<>();
stack.push(10);       // [10]
stack.push(20);       // [10, 20]
stack.push(30);       // [10, 20, 30]
stack.peek();         // 30  (top element, no removal)
stack.pop();          // 30  (remove and return top)
stack.isEmpty();      // false
stack.size();         // 2

// PREFERRED modern alternative: ArrayDeque as Stack
Deque<Integer> st = new ArrayDeque<>();
st.push(10);          // addFirst
st.peek();            // peekFirst
st.pop();             // removeFirst

// Example: Balanced Brackets Checker
public static boolean isBalanced(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c=='(' || c=='[' || c=='{') stack.push(c);
        else if (c==')') { if(stack.isEmpty()||stack.pop()!='(') return false; }
        else if (c==']') { if(stack.isEmpty()||stack.pop()!='[') return false; }
        else if (c=='}') { if(stack.isEmpty()||stack.pop()!='{') return false; }
    }
    return stack.isEmpty();
}
```

7.4 Queue and Deque

📖 WHEN TO USE

FIFO structure. Uses: BFS traversal, task scheduling, sliding window (Deque).

```
// Queue interface (FIFO)
Queue<Integer> queue = new ArrayDeque<>();
queue.offer(10);      // enqueue [10]
queue.offer(20);      // [10, 20]
queue.peek();         // 10  (front, no removal)
queue.poll();         // 10  (remove and return front)
queue.isEmpty();      // false

// Deque (double-ended queue) — can add/remove from both ends
Deque<Integer> deque = new ArrayDeque<>();
deque.addFirst(10);   // [10]
deque.addLast(20);    // [10, 20]
deque.addFirst(5);    // [5, 10, 20]
deque.peekFirst();    // 5
deque.peekLast();     // 20
deque.pollFirst();    // 5  -> [10, 20]
deque.pollLast();     // 20 -> [10]
```

7.5 PriorityQueue (Heap)

📖 WHEN TO USE

When you always need the smallest (min-heap) or largest (max-heap) element in O(log n).

```
// Min-Heap (default) — poll() returns SMALLEST element
PriorityQueue<Integer> minPQ = new PriorityQueue<>();
minPQ.offer(5);
minPQ.offer(1);
minPQ.offer(3);
minPQ.poll();     // 1  (smallest)
minPQ.peek();     // 3  (next smallest, no removal)

// Max-Heap — poll() returns LARGEST element
PriorityQueue<Integer> maxPQ = new PriorityQueue<>(Collections.reverseOrder());
// Or: new PriorityQueue<>((a,b) -> b-a);
maxPQ.offer(5); maxPQ.offer(1); maxPQ.offer(3);
maxPQ.poll();     // 5  (largest)

// Common application: Find K largest elements
public static int[] kLargest(int[] arr, int k) {
    PriorityQueue<Integer> minH = new PriorityQueue<>();
```

```
    for (int x : arr) {
        minH.offer(x);
        if (minH.size() > k) minH.poll(); // remove smallest
    }
    int[] result = new int[k];
    for (int i=k-1; i>=0; i--) result[i] = minH.poll();
    return result;
}
```

7.6 HashMap and HashSet

📖 WHEN TO USE

HashMap: O(1) key-value lookup/insert. HashSet: O(1) membership test, uniqueness enforcement.

```
// HashMap<K,V>
Map<String,Integer> map = new HashMap<>();
map.put("alice", 90);
map.put("bob",   85);
map.get("alice");                       // 90
map.getOrDefault("charlie", 0);         // 0  (key not present)
map.containsKey("bob");                 // true
map.containsValue(85);                  // true
map.remove("bob");
map.size();                             // 1

// Iterate over entries:
for (Map.Entry<String,Integer> e : map.entrySet())
    System.out.println(e.getKey() + " -> " + e.getValue());

// Useful one-liner for frequency count:
map.merge(key, 1, Integer::sum); // increment count by 1

// HashSet<E>
Set<Integer> set = new HashSet<>();
set.add(1); set.add(2); set.add(1);  // {1,2}  — no duplicates
set.contains(1);                     // true
set.remove(1);                       // {2}
set.size();                          // 1
```

7.7 TreeMap and TreeSet — Sorted Collections

```
// TreeMap — sorted by keys, O(log n) operations
TreeMap<Integer,String> tmap = new TreeMap<>();
tmap.put(3,"c"); tmap.put(1,"a"); tmap.put(2,"b");
tmap.firstKey();      // 1  (smallest key)
```

```
tmap.lastKey();       // 3  (largest key)
tmap.floorKey(2);     // 2  (largest key <= 2)
tmap.ceilingKey(2);   // 2  (smallest key >= 2)
tmap.headMap(2);      // {1=a}  keys < 2
tmap.tailMap(2);      // {2=b, 3=c}  keys >= 2

// TreeSet — sorted unique elements
TreeSet<Integer> tset = new TreeSet<>();
tset.add(5); tset.add(2); tset.add(8);
tset.first();         // 2
tset.last();          // 8
tset.floor(6);        // 5  (largest element <= 6)
tset.ceiling(3);      // 5  (smallest element >= 3)
```

7.8 Common Collection Patterns

7.8.1 Frequency Count with HashMap

```
public static Map<Integer,Integer> frequency(int[] arr) {
    Map<Integer,Integer> freq = new HashMap<>();
    for (int x : arr) freq.put(x, freq.getOrDefault(x,0)+1);
    return freq;
}
```

7.8.2 Group Anagrams using HashMap

```
public static Map<String,List<String>> groupAnagrams(String[] words) {
    Map<String,List<String>> groups = new HashMap<>();
    for (String w : words) {
        char[] c = w.toCharArray(); Arrays.sort(c);
        String key = new String(c);           // sorted form = anagram key
        groups.computeIfAbsent(key, k->new ArrayList<>()).add(w);
    }
    return groups;
}
// ["eat","tea","tan","ate","nat","bat"] ->
// {"aet":[eat,tea,ate], "ant":[tan,nat], "abt":[bat]}
```

7.8.3 Two Sum Problem using HashMap

```
public static int[] twoSum(int[] arr, int target) {
    Map<Integer,Integer> seen = new HashMap<>();
    for (int i=0; i<arr.length; i++) {
        int complement = target - arr[i];
        if (seen.containsKey(complement))
            return new int[]{seen.get(complement), i};
        seen.put(arr[i], i);
```

```
    }
    return new int[]{};  // no pair found
}
// twoSum({2,7,11,15}, 9) -> {0,1}  (2+7=9)
```

7.8.4 Sliding Window Maximum using Deque

```
// Find max in every window of size k  — O(n) total
public static int[] slidingWindowMax(int[] arr, int k) {
    Deque<Integer> dq = new ArrayDeque<>(); // stores INDICES
    int n=arr.length;
    int[] result = new int[n-k+1];

    for (int i=0; i<n; i++) {
        // Remove indices outside window
        while (!dq.isEmpty() && dq.peekFirst() <= i-k) dq.pollFirst();
        // Remove smaller elements from back
        while (!dq.isEmpty() && arr[dq.peekLast()] < arr[i]) dq.pollLast();
        dq.addLast(i);
        if (i >= k-1) result[i-k+1] = arr[dq.peekFirst()];
    }
    return result;
}
// {1,3,-1,-3,5,3,6,7}, k=3 -> {3,3,5,5,6,7}
```

7.8.5 LRU Cache Concept (LinkedHashMap)

```
// LinkedHashMap with access-order = true implements LRU cache
int capacity = 3;
Map<Integer,Integer> lru = new LinkedHashMap<>(16, 0.75f, true) {
    protected boolean removeEldestEntry(Map.Entry<Integer,Integer> e) {
        return size() > capacity;
    }
};
lru.put(1,10); lru.put(2,20); lru.put(3,30);
lru.get(1);    // access key 1 -> moves to front
lru.put(4,40); // evicts key 2 (least recently used)
```

WORKED EXAMPLES — Sample Questions & Answers

Q1: Find the first non-repeating element in {9,4,9,6,7,4}.

Use LinkedHashMap to preserve order. freq: 9->2, 4->2, 6->1, 7->1. First with count=1: 6.

Q2: Find K=2 largest elements in {3,2,1,5,6,4}.

Use min-heap of size K. After all elements: heap={5,6}. Result: [5,6]. Answer: 5 and 6.

Q3: Check if {2,7,11,15} has a pair summing to 9.

twoSum approach: i=0,x=2,complement=7,not in map;i=1,x=7,complement=2,found at 0! Return {0,1}.

✏ PRACTICE QUESTIONS — Try These Yourself

37. Using a Stack, evaluate the expression: push 3, push 5, pop two and push their sum. What is the top of the stack?

38. Use a PriorityQueue to sort {4,1,3,2,5} by dequeuing one at a time. What order do you get?

39. Group these words by anagram: ["eat","tea","tan","ate"]. Show the HashMap structure.

40. Find the two numbers that sum to 12 in {1,5,3,7,9,4}. Use HashMap.

41. Given a stream of numbers, maintain a sorted order using TreeSet and print them after inserting {5,2,8,1,9,2,3}.

✔ SOLUTIONS TO PRACTICE QUESTIONS

S1: push(3),push(5),pop()->5,pop()->3,push(3+5=8). Top=8.

S2: Min-heap dequeue order: 1,2,3,4,5 (ascending).

S3: sort "eat"->"aet", "tea"->"aet", "tan"->"ant", "ate"->"aet". Map: {aet:[eat,tea,ate], ant:[tan]}.

S4: scan: x=1,need 11(no); x=5,need 7(no); x=3,need 9(no); x=7,need 5(YES at index 1). Answer: 5 and 7.

S5: TreeSet automatically removes dup 2. After all inserts: {1,2,3,5,8,9}.

Chapter 8: Quick Reference & Exam Strategy

8.1 Time Complexity Cheat Sheet

Operation / Algorithm	Best	Average	Worst	Space
Linear scan	O(n)	O(n)	O(n)	O(1)
Binary search	O(1)	O(log n)	O(log n)	O(1)
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)
HashMap get/put	O(1)	O(1)	O(n)	O(n)
TreeMap get/put	O(log n)	O(log n)	O(log n)	O(n)
Fibonacci iterative	O(n)	O(n)	O(n)	O(1)
Prime check sqrt(n)	O(√n)	O(√n)	O(√n)	O(1)
Sieve of Eratosthenes	—	O(n log log n)	—	O(n)
GCD Euclidean	O(1)	O(log n)	O(log n)	O(1)
Matrix multiply m×k×n	—	O(m*k*n)	O(m*k*n)	O(m*n)

8.2 Pattern Recognition Guide

Problem Clue	Pattern to Apply	Data Structure
Find min/max/sum/average	Linear scan	int/long variable
Find if element exists	Binary search or HashSet	int[] or HashSet
Count frequency of elements	Hashing	HashMap<T,Integer>
Remove duplicates	Hashing or Two-Ptr	LinkedHashSet / sorted 2-ptr
Pair that sums to target	Hashing or Two-Ptr	HashMap or sorted+2-ptr
Subarray sum (range query)	Prefix sum	int[] prefix
Equilibrium / balance point	Prefix sum	long leftSum
Reverse / palindrome	Two-pointer	two index variables
Rotate array	Three-reversal trick	in-place
Sort by frequency	Sort with comparator	HashMap + sort
All primes up to N	Sieve	boolean[]
K largest/smallest	Heap	PriorityQueue

Sliding window max/min	Monotonic Deque	ArrayDeque
Balanced brackets	Stack	Deque<Character>
Level-order traversal	BFS	Queue
Group by common property	HashMap<key,List>	HashMap
Sorted unique elements	TreeSet / sort	TreeSet

8.3 Java Input / Output Template

```
import java.util.*;

public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Read a single integer
        int n = sc.nextInt();

        // Read an array
        int[] arr = new int[n];
        for (int i=0; i<n; i++) arr[i] = sc.nextInt();

        // Read a string (one word)
        String word = sc.next();

        // Read a full line (call sc.nextLine() after nextInt if needed)
        sc.nextLine();           // consume leftover newline
        String line = sc.nextLine();

        // Read a 2D matrix
        int r = sc.nextInt(), c = sc.nextInt();
        int[][] mat = new int[r][c];
        for (int i=0;i<r;i++) for(int j=0;j<c;j++) mat[i][j]=sc.nextInt();

        // Output
        System.out.println(result);
        System.out.printf("%.2f%n", avg);  // formatted decimal
        System.out.println(Arrays.toString(arr));
    }
}
```

8.4 Critical Edge Cases by Topic

Topic	Critical Edge Cases to Handle

Arrays	Empty array; single element; all same; all negative; k>=n for rotation
Strings	Empty string; single char; all same chars; leading/trailing spaces
Numbers	n=0 or n=1; negative input; overflow (use long); n=2 for prime
Number Sys	Input is "0"; leading zeros in binary/octal; single digit
Sorting	Already sorted (bubble/insertion benefit); reverse sorted (quicksort risk)
Matrices	1×1 matrix; non-square for transpose; zero matrix for multiplication
Collections	Empty collection; single element; null keys (HashMap allows one null key)

8.5 Common Mistakes and Fixes

⚠ TOP 10 CODING MISTAKES IN TCS NQT

1. Using == for String comparison: ALWAYS use .equals()
2. Integer overflow in sum/product: use long for large values
3. Off-by-one in binary search: use l <= r, mid = l+(r-l)/2
4. Accessing arr[i-1] or arr[i+1] without bounds check
5. Forgetting k = k%n before rotation
6. Using recursive Fibonacci for large n (stack overflow, O(2^n))
7. String += in a loop: always use StringBuilder
8. Scanner.nextLine() after nextInt() skipping a line
9. HashMap.get() returning null: use getOrDefault(key, 0)
10. Not handling negative numbers in digit-extraction loops

8.6 30-Second Problem Classification Checklist

READ → CLASSIFY → SOLVE

Step 1 — What is the INPUT? (single number, array, string, matrix, 2 values)
Step 2 — What is the OUTPUT? (boolean, number, array, string, list)
Step 3 — Key words:

```
'palindrome' -> reverse and compare
'prime'      -> sqrt(n) trial division or sieve
'frequency'  -> HashMap
'sorted'     -> binary search, two-pointer
'duplicate'  -> HashSet or sort
'rotate'     -> three-reversal trick
'sort by'    -> Comparator with sort
'range sum'  -> prefix array
```

Step 4 — Handle edge cases first.
Step 5 — Code, dry-run on example, submit.